

Relatório de Segurança Informática e nas Organizações

Universidade de Aveiro

Caroline Ribeiro, Ellen Sales, Julia Abrantes



Relatório de Segurança Informática e nas Organizações

Dept. de Eletrónica, Telecomunicações e Informática

Universidade de Aveiro

Caroline Ribeiro, Ellen Sales, Julia Abrantes

(106093) caroline.ribeiro@ua.pt, (117450) ellensales@ua.pt, (104170) julia.abrantes@ua.pt

30 de dezembro de 2024

Índice

1	Relatório	1
1.1	Introdução	1
1.2	Implementação	1
1.2.1	Comandos Locais	1
1.2.2	API Anônima	2
1.2.3	API Autenticada	3
1.2.4	API Autorizada	6
2	Análise do Software	11
2.1	Introdução	11
2.2	(6.1) Dados classificados	11
2.2.1	V6.1.1 - Encriptação de dados sensíveis	11
2.2.2	V6.1.2 - Privacidade de dados médicos	11
2.2.3	V6.1.3 - Privacidade de dados financeiros	12
2.3	(6.2) Algoritmos	12
2.3.1	V6.2.1 - Falhas Seguras e Prevenção de Padding Oracle	12
2.3.2	V6.2.2 - Uso de Algoritmos e Bibliotecas Criptográficas Comprovadas	13
2.3.3	V6.2.3 - Configuração Segura de Vetores de Inicialização e Modos	14
2.3.4	V6.2.4 - Agilidade diante de quebras criptográficas	16
2.3.5	V6.2.5 - Evitar Algoritmos Fracos ou Obsoletos	17
2.3.6	V6.2.6 - Não repetição de números de uso único	18
2.3.7	V6.2.7 - Autenticação de Dados Criptografados	19
2.3.8	V6.2.8 - Operações de tempo constante	21
2.4	(6.3) Random Values	22
2.4.1	Critério 6.3.1	22
2.4.2	Critério 6.3.2	22
2.4.3	Critério 6.3.3	23
2.5	(6.4) Secret Management	23
2.5.1	Critério 6.4.1	23

Capítulo 1

Relatório

1.1 Introdução

O seguinte projeto trata-se de um repositório seguro para o armazenamento e gerenciamento de documentos dentro de organizações. Cada documento é composto por um arquivo criptografado e seus metadados, que incluem informações como o nome, criador e controles de acesso. O repositório permite o gerenciamento das permissões de acesso através de listas de controle de acesso (ACLs) para documentos e organizações. Além disso, inclui a criação de sessões de segurança para interação com o repositório, autenticação de usuários e manipulação de papéis e permissões dentro das organizações. A implementação foca na segurança, confidencialidade e integridade dos dados, utilizando criptografia para proteger os arquivos e assegurar que apenas usuários autorizados possam acessar ou modificar os documentos.

1.2 Implementação

1.2.1 Comandos Locais

- **Criar credenciais**

A implementação do `rep_subject_credentials` foi feita da seguinte forma:

1. Lado do cliente

Recebe os seguintes parâmetros: *password* e *credentials_file*.

Gera o par de chaves ECC do subject com a função `generate_key_par`, que retorna a **chave privada** e a **chave pública** do subject.

Salva a chave pública do cliente em um arquivo (*credentials_file*) com a função `save_pub_key`.

2. Lado do servidor

Não se aplica.

- **Desencriptar um arquivo**

A implementação do `rep_decrypt_file` foi feita da seguinte forma:

1. Lado do cliente

O arquivo *rep_decrypt_file* é um script que descriptografa um arquivo criptografado usando metadados de criptografia fornecidos. Ele utiliza as funções auxiliares definidas no arquivo *decipher.py*:

- **parse_args**: Analisa os argumentos da linha de comando para obter o nome do arquivo criptografado, o arquivo de metadados de criptografia, o nome do arquivo descriptografado (opcional) e uma flag para determinar se a saída deve ser em bytes.
- **decrypt_file**: Descriptografa o arquivo usando os metadados fornecidos. Esta função lê o arquivo criptografado e os metadados, extrai os parâmetros de criptografia (chave, algoritmo, modo, IV, hash), e usa a função

- **decipherCBC** para descriptografar o arquivo. Também verifica a integridade do arquivo descriptografado com a função **check_hash** Argumentos:
 - * **encrypted_file**: Arquivo a ser descriptografado.
 - * **encryption_metadata**: Arquivo com os metadados de criptografia.
 - * **verbose**: Aumenta a verbosidade do log.
 - * **out_bytes**: Exibe a saída em bytes.
 - * **decrypted_file_name**: Nome do arquivo para salvar o conteúdo descriptografado.
- **save_file**: Salva o conteúdo descriptografado em um arquivo, se um nome de arquivo for fornecido.
- **main**: Função principal que chama **parse_args**, **decrypt_file** e **save_file** conforme necessário. Se a descriptografia for bem-sucedida, o conteúdo descriptografado é enviado para a saída padrão (stdout) ou salvo em um arquivo.
- **check_hash**: Verifica a integridade do arquivo descriptografado comparando o hash calculado com o hash esperado. Argumentos:
 - * **hash_expected**: o hash esperado no fim.
 - * **plaintext**: o texto plano que foi submetido à função de hash.
 - * **hash**: tipo de hash usado, por exemplo: 'sha3-224'.
- Lado do servidor
Não se aplica

1.2.2 API Anônima

– Criar uma organização

A implementação do **rep_create_org** foi feita da seguinte forma:

1. Lado do cliente

Se o caminho para a chave pública ou o endereço do repositório não estiverem configurados, é possível adicioná-los adicionando os seguintes argumentos ao fim do comando **-k <caminho>** e **-r <endereço>**. Durante a execução do comando, se tais parâmetros forem adicionados, faz um *override* se já foram declarados anteriormente ou simplesmente adiciona-os ao estado global. Também verifica se o caminho da chave é válido.

A função recebe os seguintes **parâmetros**: *organization*, *username*, *name*, *email* e *public_key_file*. E então faz as verificações se a chave pública existe e se o e-mail é válido, se alguma verificação não for válida, faz **sys.exit(-1)**.

Cria a variável **data** com os valores: *organization*, *username*, *name*, *email*, *public_key*. Utiliza a função **encrypt_no_session_message** que recebe como parâmetros a mensagem (*data*) e a chave pública do repositório e criptografa com AES a mensagem com uma chave simétrica. Retorna a **encrypted_message** - que contém a **encrypted_key** (chave simétrica encriptada com a chave pública do servidor), **encrypted_data** (mensagem encriptada com a chave simétrica), **nonce** e **tag** - e **symmetric_key** (chave simétrica gerada).

2. Lado do servidor

Descripta a **encrypted_key** com a chave privada, e depois descripta a mensagem a chave simétrica descriptada. Adiciona a organização à tabela **organizations** na base de dados com o **username** como manager e com todas as permissões. Por fim, adiciona o subject na tabela **subject** com status *"up"*. Retorna para o cliente uma mensagem de sucesso, caso não haja erros. Caso já exista uma organização com o mesmo nome, ou haja um erro, retorna um insucesso para o cliente com a respetiva mensagem.

– Listar organizações

A implementação do **rep_list_orgs** foi feita da seguinte forma:

1. Lado do cliente

Como não recebe parâmetros, espera a resposta do servidor. Quando a recebe, verifica a assinatura do repositório com **verify_repository_signature** que recebe como parâmetros

a mensagem, a assinatura e a chave pública do servidor, essa função é responsável por garantir a integridade da mensagem retornada ao cliente. Se a assinatura for válida, utiliza `logger` para listar as organizações.

2. Lado do servidor

Acede à tabela `organizations` da base de dados, guarda-os em uma variável `organizations` que é adicionada a `data` e assina o `data` com sua chave privada. Envia para o cliente um `data_to_send` que contém o `data` e `signature`.

– Criar uma sessão

A implementação do `rep_create_session` foi feita da seguinte forma:

1. Lado do cliente

Assim como utilizado em criar uma organização 1, faz a verificação do caminho da chave e/ou endereço do repositório.

Recebe os seguintes **parâmetros**: `organization`, `username`, `password`, `credentials_file` e `session_file`.

O cliente envia para o servidor um `data_to_send` que contém: `organization`, `username` e `public_key`. Então utiliza a função `encrypt_no_session_message` 1 e recebe o `encrypted_payload` e `session_key`. Assina a mensagem com a função `sign_message` - que recebe por parâmetros a `password` e a mensagem, e utiliza a `generate_key_pair` 1 para carregar a chave privada e assinar a mensagem, retornando a assinatura. Por fim, o cliente envia para o servidor o `encrypted_payload` e `signature`.

2. Lado do servidor

Descriptografa a chave simétrica com a chave pública do servidor e por fim descriptografa os dados. Verifica se a organização existe e está associada ao usuário. Então utiliza a chave pública (função `get_public_key` que com base no `username` obtém a chave pública guardada na base de dados) para verificar a assinatura do subject, utilizando a função `verify_subject_signature` - que recebe a chave pública, a mensagem e a assinatura e retorna se ela é válida. Se a assinatura for válida, então cria um ficheiro de sessão que possui o padrão `session_id.json` e nele contém o `session_data` com `user_id`, `symmetric_key`, `organization_id` e `timeout`.

– Download de ficheiro

O `rep_get_file` funciona da seguinte forma:

1. Lado do cliente

O cliente chama a função `parse_args()` para analisar os argumentos da linha de comando e ajustar o estado usando variáveis de ambiente e argumentos. Depois constrói a URL do arquivo no servidor usando o endereço do repositório e o identificador do arquivo. O cliente faz uma requisição GET para a URL construída, recebe a resposta do servidor em modo stream e salva o conteúdo do arquivo localmente, se um nome de arquivo local foi fornecido.

2. Lado do servidor

O servidor define a rota `/get_file` para lidar com requisições GET. E se encontrar o ficheiro com o nome fornecido no parâmetro, responde com o ficheiro usando a função `send_from_direct` do flask. Se o arquivo não for encontrado, o servidor retorna uma resposta JSON com um erro 404.

1.2.3 API Autenticada

– Assumir role da sessão

A implementação do `rep_assume_role` foi feita da seguinte forma:

1. Lado do cliente

Recebe os seguintes parâmetros: `session_file` e `role`.

Acessa a chave da sessão e o id da sessão com a função `get_session_data`, que recebe o `session_file` como parâmetro e retorna a chave e o id da sessão.

Encripta o role pedido com a chave da sessão com a função *encrypt_message* que recebe como parâmetro os dados e a chave da sessão e retorna os mesmos criptografados.

Envia para o servidor o role criptografado e o id da sessão.

2. Lado do servidor

O servidor recebe os dados e o id da sessão e verifica se a sessão ainda está ativada com a função *verify_session_file* que retorna true se o tempo da sessão ainda não expirou e se verifica que já expirou, a própria função apaga o *session_file* da pasta de sessões.

A partir do *session_file* o servidor acessa todas as informações necessárias para descriptografar os dados enviados.

Descriptografa os dados com a função *decrypt_message* que recebe como parâmetro os dados criptografados e chave da sessão e retorna os dados descriptografados.

Verifica na base de dados se o username da sessão está na lista de subjects no role da acl da organização.

Se sim, adiciona o role a lista de roles da sessão no *session_file*.

Retorna o role assinado com a chave privada do servidor para verificação do sucesso da operação do lado do cliente com a função *verify_repository_signature* 1

– Remover role da sessão

A implementação do *rep_drop_role* foi feita da seguinte forma:

1. Lado do cliente

Recebe os seguintes parâmetros: *session_file* e *role*.

Acessa a chave da sessão e o id da sessão com a função *get_session_data*1.

Encripta o role pedido com a chave da sessão com a função *encrypt_message* 1 e envia para o servidor o role criptografado e o id da sessão.

2. Lado do servidor

O servidor recebe os dados e o id da sessão e verifica se a sessão ainda está ativada com a função *verify_session_file*2.

A partir do *session_file* o servidor acessa todas as informações necessárias para descriptografar os dados enviados.

Descriptografa os dados com a função *decrypt_message* 2.

E remove da lista de roles da sessão no *session_file* se o username estiver com aquele role.

Retorna o role assinado com a chave privada do servidor para verificação do sucesso da operação do lado do cliente com a função *verify_repository_signature* 1 e a respectiva lista de roles.

– Listar as roles da sessão

A implementação do *rep_list_roles* foi feita da seguinte forma:

1. Lado do cliente

Recebe os seguintes parâmetros: *session_file*.

Acessa a chave da sessão e o id da sessão com a função *get_session_data*1.

Envia o id da sessão para o servidor.

2. Lado do servidor

O servidor recebe o id da sessão e verifica se a sessão ainda está ativada com a função *verify_session_file*2.

A partir do *session_file* o servidor busca a lista de roles.

Retorna o id da sessão assinado com a chave privada do servidor para verificação do sucesso da operação do lado do cliente com a função *verify_repository_signature* 1e a respectiva lista de roles.

– Listar os subjects da organização

A implementação do *rep_list_subjects* foi feita da seguinte forma:

1. Lado do cliente

Recebe os seguintes parâmetros: *session_file* e como opção [*username*].

Acessa a chave da sessão e o id da sessão com a função *get_session_data*1.

Criptografa o username caso seja passado com a chave da sessão.

Envia o id da sessão para o servidor e o dado criptografado se existir com a função *encrypt_message* 1.

2. Lado do servidor

O servidor recebe o dado e o id da sessão verifica se a sessão ainda está ativada com a função *verify_session_file*2.

Descriptografa o username se existir com a chave da sessão com a função *decrypt_message* 2.

A partir do *session_file* o servidor acessa o id da organização do e busca na base de dados do subjects todos os subjects que tenham o mesmo id caso não tenha sido passado um username como parâmetro.

Ou busca o username passado com o id da organização.

Retorna o id da sessão assinado com a chave privada do servidor para verificação do sucesso da operação do lado do cliente com a função *verify_repository_signature* 1 e a respetiva lista de subjects e seus status ou somente o username e seu status.

– **Listar os subjects em uma das roles da organização**

A implementação do *rep_list_role_subjects* foi feita da seguinte forma:

1. Lado do cliente

Recebe os seguintes parâmetros: *session_file* e *role*.

Acessa a chave da sessão e o id da sessão com a função *get_session_data*1.

Envia o role criptografado com a chave da sessão com a função *encrypt_message* 1 e o id da sessão para o servidor.

2. Lado do servidor

O servidor recebe os dados e o id da sessão e verifica se a sessão ainda está ativada com a função *verify_session_file*2.

A partir do *session_file* o servidor acessa o id da organização do username e a chave da sessão.

Descripta os dados enviados com a chave da sessão com a função *decrypt_message* 2 e busca na base de dados a lista de subjects do role da acl da organização do username ativo.

Retorna o role assinado com a chave privada do servidor para verificação do sucesso da operação do lado do cliente com a função *verify_repository_signature* 1 e a respetiva lista de subjects do role.

– **Listar as permissões em uma das roles da organização**

A implementação do *rep_list_role_permissions* foi feita da seguinte forma:

1. Lado do cliente

Recebe os seguintes parâmetros: *session_file* e *role*.

Acessa a chave da sessão e o id da sessão com a função *get_session_data*1.

Envia o role criptografado com a chave da sessão com a função *encrypt_message* 1 e o id da sessão para o servidor.

2. Lado do servidor

O servidor recebe os dados e o id da sessão e verifica se a sessão ainda está ativada com a função *verify_session_file*2.

A partir do *session_file* o servidor acessa o id da organização do username e a chave da sessão.

Descripta os dados enviados com a chave da sessão com a função *decrypt_message* 2 e busca na base de dados a lista de permissions do role da acl da organização do username ativo.

Retorna o role assinado com a chave privada do servidor para verificação do sucesso da operação do lado do cliente com a função *verify_repository_signature* 1 e a respetiva lista de permissions do role.

– **Listar as roles de um subject da organização**

A implementação do *rep_list_subject_roles* foi feita da seguinte forma:

1. Lado do cliente

Recebe os seguintes parâmetros: *session_file* e *username*.

Acessa a chave da sessão e o id da sessão com a função *get_session_data1*.

Envia o username criptografado com a chave da sessão com a função *encrypt_message* 1 e o id da sessão para o servidor.

2. Lado do servidor

O servidor recebe os dados e o id da sessão e verifica se a sessão ainda está ativada com a função *verify_session_file2*.

A partir do *session_file* o servidor acessa o id da organização do username e a chave da sessão. Descripta os dados enviados com a chave da sessão com a função *decrypt_message* 2 e busca na base de dados o username na lista de subjects de cada role da acl da organização. Retorna o username assinado com a chave privada do servidor para verificação do sucesso da operação do lado do cliente com a função *verify_repository_signature* 1 e a respetiva lista de roles do subject.

– **Listar roles que possuem dada permissão**

A implementação do *rep_list_permission_roles* foi feita da seguinte forma:

1. Lado do cliente

Recebe os seguintes parâmetros: *session_file* e *permission*.

Acessa a chave da sessão e o id da sessão com a função *get_session_data1*.

Envia a permission criptografada com a chave da sessão com a função *encrypt_message* 1 e o id da sessão para o servidor.

2. Lado do servidor

O servidor recebe os dados e o id da sessão e verifica se a sessão ainda está ativada com a função *verify_session_file2*.

A partir do *session_file* o servidor acessa o id da organização do username e a chave da sessão. Descripta os dados enviados com a chave da sessão com a função *decrypt_message* 2 e busca na base de dados a permission na lista de permissions de cada role da acl da organização. Retorna o username assinado com a chave privada do servidor para verificação do sucesso da operação do lado do cliente com a função *verify_repository_signature* 1 e a respetiva lista de roles que possuem essa permission.

– **Listar documentos**

A implementação do *rep_list_docs* foi feita da seguinte forma:

1. Lado do cliente

Recebe os seguintes parâmetros: *session_file* onde vai buscar as informações da sessão já configurada *-s subject* para filtrar por username do criador *-d date* para filtrar por data. Depois de realizar o devido parsing dos argumentos com *parse_args()*, vai buscar as informações no ficheiro de sessão. Após encriptar os dados com a chave simétrica da sessão de eleição, faz uma requisição POST para o repositório e exhibe a resposta.

2. Lado do servidor

O servidor recebe a requisição POST e chama a função *list_documents* presente no ficheiro *list_docs.py*, passando para a mesma os dados provenientes da requisição POST. A primeira ação do servidor é verificar o ficheiro da sessão e depois realizar a query necessária no banco de dados. Por fim assina os dados de filtragem que o cliente enviou, ou o id da sessão, na falta de filtragem, com a chave RSA privada do repositório, usando a função *sign_message* 1. A assinatura e os dados do documentos são enviados de volta para o utilizador, junto com um código 201 para sucesso ou 500 para erro na base de dados.

1.2.4 API Autorizada

– **Adicionar subject**

A implementação do *rep_add_subject* foi feita da seguinte forma:

1. Lado do cliente

Recebe os seguintes parâmetros: *session_file*, *username*, *name*, *email* e *credentials_file*.

Acessa a chave da sessão e o id da sessão com a função *get_session_data1*.

Carrega a chave pública do user a ser adicionado com *load_pub_key* que recebe como parâmetro o *credentials_file* e retorna a chave pública para ser usada.

Envia o username, user, email e a chave pública criptografados com a chave da sessão com a função *encrypt_message 1* e o id da sessão para o servidor.

2. Lado do servidor

O servidor recebe os dados e o id da sessão e verifica se a sessão ainda está ativada com a função *verify_session_file2*.

A partir do *session_file* o servidor acessa o id da organização do username, a chave da sessão, a lista de roles e o user da sessão.

Descripta os dados enviados com a chave da sessão com a função *decrypt_message 2*.

Verifica se o user da sessão tem permissão para adicionar um novo subject (SUBJECT_NEW) com a função *check_permission_user* que recebe como parâmetros a lista de roles da sessão, o id da organização, o user da sessão e a permissão a ser verificada. A função retorna true se o user tem um role na sua lista de roles que possui a permissão necessária e false caso não tenha.

Se tiver permissão, o servidor adiciona o username, name, email e chave pública a base de dados subjects com o mesmo id da organização da sessão.

Retorna os dados recebidos assinados com a chave privada do servidor para verificação do sucesso da operação do lado do cliente com a função *verify_repository_signature 1*.

– Ativar status do subject

A implementação do *rep_activate_subject* foi feita da seguinte forma:

1. Lado do cliente

Recebe os seguintes parâmetros: *session_file* e *username*.

Acessa a chave da sessão e o id da sessão com a função *get_session_data1*.

Envia o username criptografado com a chave da sessão com a função *encrypt_message 1* e o id da sessão para o servidor.

2. Lado do servidor

O servidor recebe os dados e o id da sessão e verifica se a sessão ainda está ativada com a função *verify_session_file2*.

A partir do *session_file* o servidor acessa o id da organização do username, a chave da sessão.

Descripta os dados enviados com a chave da sessão com a função *decrypt_message 2*.

Verifica se o user da sessão tem permissão para adicionar um novo role (SUBJECT_UP) com a função *check_permission_user2*.

Se tiver permissão, o servidor muda o status do username na base de dados subjects para UP.

Retorna os dados recebidos assinados com a chave privada do servidor para verificação do sucesso da operação do lado do cliente com a função *verify_repository_signature 1*.

– Adicionar role

A implementação do *rep_add_role* foi feita da seguinte forma:

1. Lado do cliente

Recebe os seguintes parâmetros: *session_file* e *role*

Acessa a chave da sessão e o id da sessão com a função *get_session_data1*.

Envia o role criptografado com a chave da sessão com a função *encrypt_message 1* e o id da sessão para o servidor.

2. Lado do servidor

O servidor recebe os dados e o id da sessão e verifica se a sessão ainda está ativada com a função *verify_session_file2*.

A partir do *session_file* o servidor acessa o id da organização do username e a chave da sessão.

Descripta os dados enviados com a chave da sessão com a função *decrypt_message 2*.

Verifica se o user da sessão tem permissão para adicionar um novo role (ROLE_NEW) com a função *check_permission_user2*.

Se tiver permissão, o servidor adiciona na acl da base de dados das organizações o role na organização com o mesmo id da organização da sessão.

Retorna os dados recebidos assinados com a chave privada do servidor para verificação do sucesso da operação do lado do cliente com a função *verify_repository_signature* 1.

– Suspend status da role

A implementação do `rep_suspend_role` foi feita da seguinte forma:

1. Lado do cliente

Recebe os seguintes parâmetros: *session_file* e *role*.

Acessa a chave da sessão e o id da sessão com a função *get_session_data1*.

Envia o role criptografado com a chave da sessão com a função *encrypt_message* 1 e o id da sessão para o servidor.

2. Lado do servidor

O servidor recebe os dados e o id da sessão e verifica se a sessão ainda está ativada com a função *verify_session_file2*.

A partir do *session_file* o servidor acessa o id da organização do username a chave da sessão.

Descripta os dados enviados com a chave da sessão com a função *decrypt_message* 2.

Verifica se o user da sessão tem permissão para adicionar um novo role (ROLE_DOWN) com a função *check_permission_user2*.

Se tiver permissão, o servidor muda o status do role na acl da base de dados organizations para DOWN.

Retorna os dados recebidos assinados com a chave privada do servidor para verificação do sucesso da operação do lado do cliente com a função *verify_repository_signature* 1.

– Reativar status da role

A implementação do `rep_reactivate_role` foi feita da seguinte forma:

1. Lado do cliente

Recebe os seguintes parâmetros: *session_file* e *role*.

Acessa a chave da sessão e o id da sessão com a função *get_session_data1*.

Envia o role criptografado com a chave da sessão com a função *encrypt_message* 1 e o id da sessão para o servidor.

2. Lado do servidor

O servidor recebe os dados e o id da sessão e verifica se a sessão ainda está ativada com a função *verify_session_file2*.

A partir do *session_file* o servidor acessa o id da organização do username a chave da sessão.

Descripta os dados enviados com a chave da sessão com a função *decrypt_message* 2.

Verifica se o user da sessão tem permissão para adicionar um novo role (ROLE_UP) com a função *check_permission_user2*.

Se tiver permissão, o servidor muda o status do role na acl da base de dados organizations para UP.

Retorna os dados recebidos assinados com a chave privada do servidor para verificação do sucesso da operação do lado do cliente com a função *verify_repository_signature* 1.

– Suspend status do subject

A implementação do `rep_suspend_subject` foi feita da seguinte forma:

1. Lado do cliente

Recebe os seguintes parâmetros: *session_file* e *username*.

Acessa a chave da sessão e o id da sessão com a função *get_session_data1*.

Envia o username criptografado com a chave da sessão com a função *encrypt_message* 1 e o id da sessão para o servidor.

2. Lado do servidor

O servidor recebe os dados e o id da sessão e verifica se a sessão ainda está ativada com a função *verify_session_file2*.

A partir do *session_file* o servidor acessa o id da organização do username a chave da sessão.

Descripta os dados enviados com a chave da sessão com a função *decrypt_message* 2.

Verifica se o user da sessão tem permissão para adicionar um novo role (SUBJECT_DOWN) com a função *check_permission_user2*.

Se tiver permissão, o servidor muda o status do username na base de dados subjects para DOWN.

Retorna os dados recebidos assinados com a chave privada do servidor para verificação do sucesso da operação do lado do cliente com a função *verify_repository_signature* 1.

– Adicionar um subject ou uma permissão a uma role

A implementação do `rep_add_permission` foi feita da seguinte forma:

1. Lado do cliente

Recebe os seguintes parâmetros: *session_file*, *role*, *username* ou *permission*.

Acessa a chave da sessão e o id da sessão com a função *get_session_data* 1.

Envia o role, o username ou a permission criptografados com a chave da sessão com a função *encrypt_message* 1 e o id da sessão para o servidor.

2. Lado do servidor

O servidor recebe os dados e o id da sessão e verifica se a sessão ainda está ativada com a função *verify_session_file* 2.

A partir do *session_file* o servidor acessa o id da organização do username e a chave da sessão. Descripta os dados enviados com a chave da sessão com a função *decrypt_message* 2.

Verifica se o user da sessão tem permissão para adicionar um novo subject ou permission (ROLE_MOD) com a função *check_permission_user* 2.

Se tiver permissão, o servidor adiciona na acl da base de dados das organizações o subject na lista de subjects ou a permission na lista de permissions do role na organização com o mesmo id da organização da sessão.

Retorna os dados recebidos assinados com a chave privada do servidor para verificação do sucesso da operação do lado do cliente com a função *verify_repository_signature* 1.

– Remover um subject ou permissão de uma role

A implementação do `rep_remove_permission` foi feita da seguinte forma:

1. Lado do cliente

Recebe os seguintes parâmetros: *session_file*, *role*, *username* ou *permission*.

Acessa a chave da sessão e o id da sessão com a função *get_session_data* 1.

Envia o role, o username ou a permission criptografados com a chave da sessão com a função *encrypt_message* 1 e o id da sessão para o servidor.

2. Lado do servidor

O servidor recebe os dados e o id da sessão e verifica se a sessão ainda está ativada com a função *verify_session_file* 2.

A partir do *session_file* o servidor acessa o id da organização do username e a chave da sessão. Descripta os dados enviados com a chave da sessão com a função *decrypt_message* 2.

Verifica se o user da sessão tem permissão para remover um subject ou uma permission (ROLE_MOD) com a função *check_permission_user* 2.

Se tiver permissão, o servidor remove da acl da base de dados das organizações o subject da lista de subjects ou a permission da lista de permissions do role na organização com o mesmo id da organização da sessão.

Retorna os dados recebidos assinados com a chave privada do servidor para verificação do sucesso da operação do lado do cliente com a função *verify_repository_signature* 1.

– Upload de um ficheiro

O `rep_create_doc` funciona da seguinte forma:

1. Lado do cliente

O cliente primeiramente recupera os argumentos necessários com o `parse_args()` depois encripta o ficheiro com a função `encrypt_file` e o envia já encriptado juntamente com os metadados de encriptação para o repositório. Os metadados são encriptados com a função `encrypt_message` 1 antes de serem enviados.

* `encrypt_file` é uma função presente dentro do ficheiro *cipher.py* que lê um ficheiro a partir do nome passado como argumento e aplica sobre o conteúdo a função `cipherCBC` com o iv e a chave passados como argumento

* cipherCBC

2. Lado do servidor

O servidor verifica se o ficheiro não foi alterado na transmissão da mensagem com a verificação do hash enviado pelo cliente. Após descriptar a parte json da mensagem com o `decrypt_message` 2 salva os dados de encriptação na base de dados. Porém, ele deveria primeiro verificar a sessão do cliente e as respectivas roles e permissões do cliente para adicionar um ficheiro.

– **Download de metadados do ficheiro**

1. Lado do cliente

O cliente seria responsável por enviar uma requisição POST para o servidor, com um json encriptado onde constaria o nome do documento que ele pretende recuperar os dados.

2. Lado do servidor

O servidor estaria preparado para receber tal requisição, verificar a validade da sessão, descriptar o pedido e validar se o cliente possui o role com as permissões necessárias para ler o ficheiro em questão.

– **Excluir um documento**

A implementação do `rep_delete_doc` foi feita da seguinte forma:

1. Lado do cliente

Recebe os seguintes parâmetros: *session_file* e *document_name*.

Carrega a session key e a session id do *session_file*, envia para o servidor o `data_to_send` que contém `session_id` e `encrypted_data`, que contém o nome do documento encriptografado com a função `encrypt_message` 1. Aguarda o retorno do servidor, e assim como outras funções, verifica a assinatura 1.

2. Lado do servidor

Carrega o `session_id` e verifica a sessão deste id 2, com a sessão válida, carrega os dados da sessão e então descriptografa os dados recebidos 2. Por fim, verifica as permissões do documento com função `check_document_permission` - que verifica se um usuário, identificado por seu role, tem permissão específica para acessar um documento dentro de uma organização - e se houver as permissões, obtém o documento pelo nome na base de dados e assina a mensagem de confirmação para retornar ao cliente.

– **Mudar ACL de um documento**

Recebe os seguintes parâmetros: *session_file*, *document_name*, *operation* e *permission*.

1. Lado do cliente

Carrega a `session_key` e encripta os dados (`document_name`, `operation`, `role` e `permission`) com a chave da sessão e envia para o servidor. Quando recebe o retorno, verifica a assinatura para confirmar se houve êxito.

2. Lado do servidor

Carrega o `session_id` e verifica a sessão deste id 2, com a sessão válida, carrega os dados da sessão e então descriptografa os dados recebidos 2. Por fim, verifica as permissões do documento 2 e se houver as permissões, faz as devidas alterações. Por fim, assina a mensagem de êxito e retorna para o cliente.

Capítulo 2

Análise do Software

2.1 Introdução

Foi utilizado o capítulo V6 do ASVS para a análise do projeto. O capítulo está relacionado à criptografia aplicada a dados armazenados e seu gerenciamento seguro. Este controle de segurança tem como objetivo garantir que os sistemas que utilizam criptografia para proteger dados sensíveis (como senhas, chaves e informações de autenticação) sejam projetados de maneira robusta e segura.

2.2 (6.1) Dados classificados

2.2.1 V6.1.1 - Encriptação de dados sensíveis

Objetivo: Garantir que caso haja algum vazamento de dados devido a outra vulnerabilidade, os dados sensíveis estão protegidos por encriptação e não são legíveis nem decifráveis por atacantes em posse dos mesmos.

Sugestão de melhoria: No nosso projeto não implementamos tais mecanismos, mas as informações pessoais poderiam ser encriptadas enquanto em descanso com a encriptação geral da base de dados. Para tal, poderíamos usar a biblioteca `pysqlcipher3` David Riggleman [1] que é uma interface python para o `SQLCipher` Zetetic, LLC [2]. Desta forma evitaria-se a fraqueza descrita em `CWE-316: Cleartext Storage of Sensitive Information in Memory` CWE List [3].

Exemplo de melhoria: Precisamos desta forma passar a chave da base de dados sempre que fazemos a conexão com a mesma:

```
conn = sqlite3.connect(f"{repository_path}")
conn.execute(f"PRAGMA key='{database_key}'")
cursor = conn.cursor()
```

Figura 2.1: Exemplo de como poderia ser encriptada a base de dados

2.2.2 V6.1.2 - Privacidade de dados médicos

Análise: Dado que os dados que guardamos, não são fundamentalmente de natureza médica, este tópico não se aplica ao projeto de forma genérica. Contudo, é necessário mencionar que caso al-

gum ficheiro enviado por clientes contenha dados de tal natureza, os ficheiros são de facto guardados encriptados.

2.2.3 V6.1.3 - Privacidade de dados financeiros

Análise: Dado que os dados que guardamos, não são fundamentalmente de natureza financeira, este tópico não se aplica ao projeto de forma genérica. Contudo, é necessário mencionar que caso algum ficheiro enviado por clientes contenha dados de tal natureza, os ficheiros são de facto guardados encriptados.

2.3 (6.2) Algoritmos

2.3.1 V6.2.1 - Falhas Seguras e Prevenção de Padding Oracle

Descrição: Todos os módulos criptográficos devem falhar de forma segura, e o tratamento de erros não deve vaziar informações que possam habilitar ataques de Padding Oracle.

Aplicação:

- **Falhas Seguras:** Erros em processos criptográficos são tratados com mensagens de erro genéricas para evitar o vazamento de informações sensíveis.
- **Prevenção de Padding Oracle:** AES-GCM é usado para criptografia, o que evita vulnerabilidades de preenchimento ao não depender de mecanismos de preenchimento explícito(What is AES-GCM? (Cloudflare)).

Exemplos de Código:

```
7 def decrypt_message(encrypted_payload, symmetric_key):
8     """
9     Descriptografa os dados usando uma chave simétrica compartilhada.
10
11     :param encrypted_payload: Dicionário contendo dados criptografados, nonce e tag.
12     :param shared_key: Chave simétrica compartilhada (bytes).
13     :return: Dados descriptografados (bytes).
14     """
15     try:
16         # Recupera os valores do dicionário
17         encrypted_data = bytes.fromhex(encrypted_payload["encrypted_data"])
18         nonce = bytes.fromhex(encrypted_payload["nonce"])
19         tag = bytes.fromhex(encrypted_payload["tag"])
20
21         # Configura o cifrador com AES no modo GCM
22         cipher = Cipher(
23             algorithms.AES(symmetric_key),
24             modes.GCM(nonce, tag),
25             backend=default_backend()
26         )
27         decryptor = cipher.decryptor()
28
29         # Descriptografa os dados
30         decrypted_message = decryptor.update(encrypted_data) + decryptor.finalize()
31
32         return decrypted_message
33     except Exception as e:
34         raise ValueError(f"Erro ao descriptografar os dados: {e}")
```

Figura 2.2: Tratamento de erros para evitar vazamento de dados

Se a descriptografia falhar devido a dados inválidos, chaves incorretas ou adulteração, a exceção é capturada e relançada com uma mensagem de erro genérica. Isso evita a exposição de detalhes sensíveis

sobre a falha, como estados criptográficos específicos, garantindo que os atacantes não possam explorar mensagens de erro para inferir fraquezas ou entradas válidas.

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.backends import default_backend
import os

def encrypt_message(data_to_send, symmetric_key) -> dict: #session_key no create seession vai criptografada com a chave publica do servidor
    # Gera um nonce único para esta operação de criptografia
    nonce = os.urandom(12) # Recomendado para AES-GCM

    # Configura o cifrador com AES no modo GCM
    cipher = Cipher(
        algorithms.AES(symmetric_key),
        modes.GCM(nonce),
        backend=default_backend()
    )
    encryptor = cipher.encryptor()

    # Criptografa os dados
    encrypted_data = encryptor.update(data_to_send) + encryptor.finalize()
    tag = encryptor.tag

    # Retorna os dados criptografados, nonce e tag
    encrypted_message = {
        "encrypted_data": encrypted_data.hex(),
        "nonce": nonce.hex(),
        "tag": tag.hex(),
    }

    return encrypted_message
```

Figura 2.3: Criptografando uma mensagem de forma segura usando AES-GCM

A função garante criptografia segura usando o modo AES-GCM, que evita ataques de Padding Oracle de forma inerente. O nonce único e a tag gerados para cada operação garantem a confidencialidade e a integridade dos dados criptografados.

2.3.2 V6.2.2 - Uso de Algoritmos e Bibliotecas Criptográficas Comprovadas

Descrição: Apenas algoritmos, modos e bibliotecas criptográficas comprovados ou aprovados por governos devem ser usados. Criptografia personalizada deve ser evitada.

Aplicação:

- **Seleção de Algoritmos:** AES-256 é usado para criptografia e SHA-256 é usado para hashing. Ambos são implementados via a biblioteca **cryptography** em Python (Python Cryptography Library Documentation).
- **Evitar Criptografia Personalizada:** Todas as operações criptográficas utilizam bibliotecas certificadas, como OpenSSL.

O projeto utiliza **AES-256** para criptografia simétrica, amplamente reconhecido como um algoritmo seguro e aprovado pelo governo para proteger dados sensíveis. A geração da chave de 256 bits garante máxima segurança, e sua transferência segura é realizada criptografando a chave simétrica usando **RSA com preenchimento OAEP** (NIST SP 800-38A). O esquema de preenchimento utiliza **SHA-256** para hashing seguro, garantindo tanto a confidencialidade quanto a integridade da chave simétrica durante a transmissão.

A função de descryptografia incorpora **tratamento seguro de erros** utilizando um bloco **try-except**. Isso evita o vazamento de detalhes criptográficos sensíveis que poderiam ser explorados por atacantes, garantindo conformidade com os princípios de falha segura.

Ao usar AES-GCM, a implementação evita de forma inerente vulnerabilidades associadas a esquemas de preenchimento mais antigos, como o PKCS#1 v1.5. AES-GCM integra autenticação e criptogra-

fia, garantindo a integridade do texto cifrado e a proteção contra adulterações. Cada operação de criptografia gera um **nonce** único, aumentando ainda mais a segurança.

Exemplo de Código:

```
# Função para gerar uma chave simétrica
def generate_symmetric_key():
    # Gera uma chave AES de 256 bits para comunicação segura
    return os.urandom(32)

# Encripta a chave simétrica usando a chave pública do servidor
def encrypt_symmetric_key(symmetric_key, server_public_key):
    ciphertext = server_public_key.encrypt(
        symmetric_key,
        padding.OAEP(
            mgf=padding.MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(),
            label=None
        )
    )
    return ciphertext
```

Figura 2.4: Chave simétrica AES de 256 bits e sua criptografia usando RSA com preenchimento OAEP baseado em SHA-256

2.3.3 V6.2.3 - Configuração Segura de Vetores de Inicialização e Modos

Descrição: Vetores de inicialização (IVs), modos de cifra e configurações devem ser gerados e gerenciados de forma segura.

Aplicação:

- **Geração de IVs:** O IV utilizado no AES-CBC é fixo, mas o nonce do AES-GCM é único e é gerado um novo com "os.random" toda vez que uma mensagem é encriptada.
- **Modos de Cifra Seguros:** AES-GCM é usado, pois fornece tanto criptografia quanto autenticação. E para a criptografia de arquivos, foi utilizado o AES-CBC (AES-CBC vs AES-GCM: A Practical Comparison).

Nesta implementação, AES-GCM é usado com uma chave simétrica e nonce gerados de forma segura. No entanto, enquanto o processo de criptografia usa corretamente um **nonce** único, o vetor de inicialização (IV) é explicitamente implementado como nonce. As melhores práticas recomendam tratar e documentar explicitamente o uso de IV para evitar reutilização acidental em operações de criptografia, o que pode levar a vulnerabilidades. O gerenciamento adequado de IV é fundamental para conformidade com as diretrizes do OWASP ASVS e foi implementado nesse projeto com AES-CBC e como nonce no AES-GCM.

Exemplo de Código:

AES-GCM integra criptografia e autenticação. A **tag** gerada durante a criptografia garante a integridade e a autenticidade do texto cifrado, nonce e dados associados. Se adulterações forem detectadas durante a descifragem, o processo falha, fornecendo proteção robusta contra modificações não autorizadas.

```

# função para encriptar uma mensagem sem sessão
def encrypt_no_session_message(data_to_send, repo_pub_key):

    server_public_key = serialization.load_pem_public_key(repo_pub_key.encode(), backend=default_backend())

    symmetric_key = generate_symmetric_key()
    nonce = os.urandom(12)
    cipher = Cipher(algorithms.AES(symmetric_key), modes.GCM(nonce), backend=default_backend())
    encryptor = cipher.encryptor()

    data_to_send = json.dumps(data_to_send)

    encrypted_data = encryptor.update(data_to_send.encode()) + encryptor.finalize()
    tag = encryptor.tag
    encrypted_symmetric_key = encrypt_symmetric_key(symmetric_key, server_public_key)

    encrypted_message = {
        "encrypted_key": encrypted_symmetric_key.hex(),
        "encrypted_data": encrypted_data.hex(),
        "nonce": nonce.hex(),
        "tag": tag.hex(),
    }

    return encrypted_message, symmetric_key

```

Figura 2.5: Implementação de AES-GCM sem vetor de inicialização

Como o AES-GCM é projetado para oferecer criptografia autenticada e é mais apropriado para transmissões curtas de dados ou mensagens, ele não é ideal para lidar com grandes volumes de plaintext, como arquivos extensos. Para atender a esse caso de uso, foi utilizado o AES-CBC (Cipher Block Chaining), um modo amplamente adotado e seguro para criptografar grandes volumes de dados.

O AES-CBC opera dividindo os dados em blocos de tamanho fixo e aplicando um processo de encadeamento, onde cada bloco é combinado com o bloco anterior antes de ser criptografado. Isso garante que blocos idênticos de texto plano resultem em blocos cifrados diferentes, oferecendo maior segurança.

O uso do Cipher Block Chaining foi escolhido para criptografar grandes arquivos devido à sua eficiência e compatibilidade com dados extensos. Um IV fixo foi gerado para criptografia de arquivo, e o hash do ciphertext foi calculado usando SHA3-224 para verificar a integridade dos arquivos. Essa combinação garante segurança e conformidade com os requisitos do OWASP ASVS.

```

###CIPHER CBC
def cipherCBC(key:bytes, plaintext: bytes, iv: bytes) -> bytes:
    #TODO add verification
    cipher = Cipher(
        algorithm = algorithms.AES128(key),
        mode = modes.CBC(iv)
    )
    encryptor = cipher.encryptor()
    logging.debug("plaintext: %s", plaintext)
    ciphertext = encryptor.update(pad(plaintext, AES.block_size)) + encryptor.finalize()
    logging.debug("ciphertext: %s", ciphertext)

    metadata = {
        'alg': {
            'name': 'AES-128',
            'keysize': 128,
            'mode': 'CBC',
            'iv': iv.hex(),
            'hash': 'sha3-224'
        },
        'key': key.hex()
    }

    return ciphertext, metadata

```

Figura 2.6: Implementação de AES-CBC na criptografia de arquivos

2.3.4 V6.2.4 - Agilidade diante de quebras criptográficas

Objetivo: Permitir uma rápida recuperação de eventuais quebras criptográficas, através de código flexível.

Aplicação: O nosso código utiliza:

- **SHA3** de 224 bits: para criação do `file_handle`, a partir do hash do conteúdo do ficheiro, e para verificação de integridade do mesmo.
- **SHA256**: para compactar o tamanho da mensagem antes da assinatura das usando **ECDSA** (Elliptic Curve Digital Signature Algorithm) e para **MGF1** (Mask Generation Function 1)
- **AES-GCM** para cifrar e verificar a integridade das mensagens. Usamos um *nonce* de tamanho de 96 bits, *tag* de tamanho 128 bits e chave de tamanho 256 bits.
- **AES-128** modo CBC com chave de comprimento de 128 e vetor de inicialização de tamanho igual ao bloco, ou seja, 128 bits para cifrar os ficheiros, realizando-se a verificação da integridade a partir do algoritmo de digest **SHA3** como descrito acima.
- **RSA** para as chaves assimétricas referentes ao de comprimento 1024 bits e expoente público igual a 65537. a chave privada é guardada encriptada com a “melhor encriptação disponível” do módulo cryptography 43.0.3 *pyca/cryptography* [4] em PKCS8 e a pública com PKCS1.
- **ECC** foi usado para assinatura e verificação de autenticidade das mensagens, através de **ECDSA** [5], tendo as chaves sido criadas na curva **brainpoolP256r1**.

Análise: Essas tecnologias atualmente são consideradas seguras, e no nosso código podemos atualizar estas definições através da alteração da implementação das funções que realizam criptografia e hash; Nomeadamente seriam alterados os ficheiros `cypher.py` e `decypher.py`, `encrypt_message.py` e `decrypt_message.py`, para além dos pontos de chamamento das funções e criação de chaves no `commom_functions.py` e no `repo_operations.py`.

Sugestão de melhoria: O processo de reconfiguração do esquema criptográfico do sistema poderia ser facilitado com a construção de um ficheiro de configuração, e um código apto para lê-lo e adaptar-se aos algoritmos ali escolhidos.

Exemplo de melhoria: O ficheiro de configuração, por exemplo, `config.yaml` dos algoritmos criptográficos presentes no código poderiam ser algo do género:

```
message_encryption:
  algorithm: AES
  mode: GCM
  key_size: 256
  nonce_size: 96
  tag_size: 128

file_encryption:
  algorithm: AES
  block_size: 128
  mode: CBC
  iv_size: 128
  key_size: 128

server_key:
  algorithm: RSA
  key_size: 1024
  encoding_private_key: PKCS8
  encoding_public_key: PKCS1

client_key:
  algorithm: ECC
  curve: brainpoolP256r1
  public_exponent: 65537
  public_key_encoding: PEM
```

Figura 2.7: Exemplo de ficheiro de configuração dos algoritmos

2.3.5 V6.2.5 - Evitar Algoritmos Fracos ou Obsoletos

Descrição: Modos de bloco inseguros (por exemplo, ECB), mecanismos de preenchimento inseguros (por exemplo, PKCS#1 v1.5) e algoritmos de hash fracos (por exemplo, MD5, SHA-1) não devem ser usados.

Aplicação:

- Algoritmos obsoletos como MD5 e SHA-1 são evitados. SHA-256 é usado para hashing(NIST SP 800-38D).
- O modo ECB não é usado. Em vez disso, AES-GCM, um modo autenticado seguro, é implementado, assim como ECC.

A implementação garante o uso de algoritmos criptográficos fortes e evita métodos obsoletos, alinhando-se totalmente aos requisitos do OWASP ASVS V6.2.5. Na função de descriptografia fornecida para chaves simétricas, é utilizado o RSA com **preenchimento OAEP**. Este esquema de preenchimento incorpora **SHA-256** para hashing, garantindo que o processo de descriptografia seja seguro e resistente a ataques de Padding Oracle.

Além disso, é utilizado AES-GCM, um modo de cifra moderno e seguro, para criptografia simétrica. AES-GCM integra tanto criptografia quanto autenticação, fornecendo confidencialidade e integridade. Também foi usado o AES-CBC para criptografar ficheiros, já que o AES-GCM não é ideal para plaintext de grandes volumes. O AES-CBC, quando configurado corretamente com um vetor de inicialização (IV) único, é uma alternativa segura para esse tipo de uso.

A implementação também faz uso da **Criptografia de Curvas Elípticas (ECC)** para gerar assinaturas digitais e autenticar mensagens. O ECC é reconhecido como uma alternativa moderna e eficiente ao RSA, fornecendo níveis equivalentes de segurança com tamanhos de chave significativamente menores, o que melhora o desempenho e reduz os requisitos de armazenamento.

A combinação de RSA com preenchimento OAEP, AES-GCM, AES-CBC e ECC evita algoritmos inseguros como o preenchimento PKCS#1 v1.5, o modo ECB ou funções de hash desatualizadas como MD5 e SHA-1, que não são mais consideradas seguras.

Exemplo de Código:

```
# Função para decryptar a chave simétrica com a chave privada do servidor
def decrypt_symmetric_key(encrypted_symmetric_key, server_private_key):
    encrypted_symmetric_key = bytes.fromhex(encrypted_symmetric_key)
    plaintext = server_private_key.decrypt(
        encrypted_symmetric_key,
        padding.OAEP(
            mgf=padding.MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(),
            label=None
        )
    )
    return plaintext
```

Figura 2.8: Descriptografia RSA de chaves simétricas

2.3.6 V6.2.6 - Não repetição de números de uso único

Aplicação AES-CBC: Neste caso, tanto a chave quanto o IV são gerados com aleatoriedade criptográfica para cada encriptação de ficheiro, nunca havendo reutilização de chaves. Portanto configura-se aleatoriedade suficiente para que o mesmo texto não seja encriptado com o mesmo par chave/iv.

Aplicação AES-GCM: Como descrito na secção de justificação do tópico OWASP 6.2.4, o tamanho está apropriado para os critérios mais recentes de segurança, enquanto como provamos na secção da

OWASP 6.3.1 os métodos também é criptograficamente seguro, e um nível elevado de randomicidade é garantido.

Sugestão de melhoria: Por outro lado, não foi implementado nenhum controlo com relação aos nonces já utilizados. Portanto, para a encriptação das mensagens trocadas por AES Galois Counter Mode, a melhor prática seria seguir a recomendação NIST: “The total number of invocations of the authenticated encryption function shall not exceed 2^{32} , including all IV lengths and all instances of the authenticated encryption function with the given key. [...] For the RBG-based construction of IVs, the above requirement, in conjunction with the requirement that $r(i) \geq 96$, is sufficient to ensure the uniqueness requirement [...]” Dworkin [6]. Estabelecendo-se a morte da sessão após 1 hora de uso, ou se ocorrer primeiro, 2^{32} encriptações usando a mesma chave de sessão.

Exemplo de melhoria: O número de mensagens trocadas na sessão poderia ser guardado do lado do repositório, no próprio ficheiro da sessão para que seus valores possam ser consultados e validados devidamente, respeitando o limite de 1 hora ou 2^{32} encriptações, como demonstra o exemplo a seguir:

```
{
  "user_id": 1,
  "symmetric_key": "8bb849f2e55e5d06117ac1bd1bde3a598a9a6a21c0088875cbecdc10200b6d7",
  "organization_id": [1],
  "timeout": 1734829081.6288164,
  "remaining_attempts": 4294967296
}
```

Figura 2.9: Exemplo de como acrescentaríamos o limite de utilizações no ficheiro de sessão do lado servidor

2.3.7 V6.2.7 - Autenticação de Dados Criptografados

Descrição: Os dados criptografados devem ser autenticados usando assinaturas, modos de cifra autenticados ou HMACs para garantir a integridade.

Aplicação:

- **Criptografia Autenticada:** AES-GCM garante tanto criptografia quanto autenticação dos dados.
- **Assinaturas Digitais:** Em casos onde é necessária verificação explícita, assinaturas digitais são usadas.

O projeto garante que todas as mensagens trocadas entre o servidor e o cliente sejam autenticadas usando assinaturas criptográficas. Isso assegura a integridade e a autenticidade dos dados, protegendo-os contra adulterações e modificações não autorizadas.

No lado do cliente, as mensagens são assinadas usando **Criptografia de Curvas Elípticas (ECC)**. A função `sign_message` utiliza ECC com o algoritmo **ECDSA** e **SHA-256** para gerar uma assinatura digital segura. Esta assinatura é incluída com a mensagem ao ser enviada.

O servidor verifica a integridade das mensagens utilizando a função `verify_subject_signature`. Isso garante que a assinatura corresponda à chave pública do remetente, confirmando tanto a autenticidade quanto a integridade dos dados recebidos.

Além das mensagens, a integridade dos ficheiros é garantida usando o **SHA3 de 224 bits**. Este algoritmo é utilizado para criar um identificador único `file_digest` a partir do hash do conteúdo criptografado do ficheiro e verificar a integridade do ficheiro, comparando o hash calculado com o armazenado previamente.

```
def encrypt_file(file_name: str, key: bytes, iv: bytes) -> None:
    logger.debug("encrypting file %s", file_name)
    #open file
    with open(file_name, 'rb') as f:
        plaintext = f.read()
        logger.debug("data: %s", plaintext)

    #generate digest

    #encrypt plaintext
    ciphertext, restricted_metadata = cipherCBC(key, plaintext, iv)
    file_digest = sha3_224(ciphertext).hexdigest()
    logger.debug("ciphertext: %s", ciphertext)

    return restricted_metadata, file_digest, ciphertext
```

Figura 2.10: Função para criptografar um ficheiro e gerar o `file_digest`

O uso do **SHA3-224** proporciona um método eficiente e seguro para calcular hashes de ficheiros, garantindo tanto integridade quanto unicidade. O hash gerado como `file_digest` é usado como identificador único e também para verificar a consistência do conteúdo do ficheiro durante as operações.

Além disso, para certas operações, o cliente utiliza assinaturas RSA para verificar a integridade das mensagens do repositório. A função `verify_repository_signature` utiliza RSA com **preenchimento PSS** e **SHA-256** para validar a assinatura.

Essa abordagem dupla garante autenticação robusta e está alinhada com as diretrizes do OWASP ASVS ao integrar ECC e RSA para o gerenciamento de assinaturas digitais.

Exemplos de Código:

```
# Função para assinar uma mensagem com ECC
# receive parameters in hex string / return in hex string
def sign_message(password: str, message: str) -> str:
    private_key, pub = generate_key_pair(password)

    signature = private_key.sign(
        message.encode(),
        ec.ECDSA(hashes.SHA256())
    )
    return signature.hex()
```

Figura 2.11: Assinatura de mensagens em ambos os lados

```

#verify ECC user signature
#receive parameters in hex format
def verify_subject_signature(public_key: str, message: str, signature) -> bool:
    # Hash dos dados antes de verificar
    signature = bytes.fromhex(signature)
    message = message.encode()
    try:
        ##to remmber: message e signature são bytes
        public_key.verify(signature, message, ec.ECDSA(hashes.SHA256()))
        print("Assinatura verificada com sucesso")
        return True
    except Exception:
        print("Invalid signature")
        return False

```

Figura 2.12: Verificação de assinatura no lado do servidor

```

#função para verificar uma assinatura RSA
def verify_repository_signature(message, signature, public_key):
    try:
        public_key.verify(
            signature,
            message,
            padding.PSS(
                mgf=padding.MGF1(hashes.SHA256()),
                salt_length=padding.PSS.MAX_LENGTH
            ),
            hashes.SHA256()
        )
        return True
    except InvalidSignature:
        return False

```

Figura 2.13: Verificação de assinatura de repositório no lado do cliente

Essas funções demonstram como ECC e RSA são usados para assinar e verificar mensagens. ECC oferece uma assinatura eficiente e segura para mensagens de usuários, enquanto RSA é utilizado para verificar assinaturas de repositórios. Esta abordagem em camadas garante que toda a comunicação entre o servidor e o cliente seja autenticada e segura.

2.3.8 V6.2.8 - Operações de tempo constante

Análise: Ao longo do código utilizamos na maioria dos casos as bibliotecas cryptography e hashlib que são projetadas para executar operações criptográficas críticas em tempo constante. Apesar disso, segundo nossa pesquisa revela, o método padding PKCS1 v1.5, que o cryptography oferece apresenta

durante a descryptografia, esta vulnerabilidade pyca/cryptography [7], que foi evitada usando um padding mais moderno, o OAEP.

Sugestão de melhoria: Na a verificação do hash do ficheiro recebido, neste caso em particular, foi implementado uma comparação de strings de hashes com o operador “!=” que, em python não é constant-time, já que interrompe no primeiro caractere que não é igual.

Exemplo de melhoria: Para corrigir o problema mencionado anteriormente, o mais adequado seria utilizar uma função de comparação projetada para ser constante no tempo como a `compare_digest` da biblioteca hmac padrão do python, como mostra a correção a seguir:

```
#if file.filename != file_digest:
if not hmac.compare_digest(file.filename, file_digest):
    return 'File hash do not coincide with name indicated', 400
```

Figura 2.14: Correção em uma das comparações realizadas que não obedece ao princípio de tempo constante

2.4 (6.3) Random Values

2.4.1 Critério 6.3.1

Descrição: Todos os números aleatórios, nomes de ficheiros aleatórios, GUIDs e strings aleatórias são gerados usando um gerador de números aleatórios criptograficamente seguro.

Aplicação: Durante a criação de números aleatórios, foi utilizado o `os.urandom`, que gera uma sequência de bytes aleatórios com qualidade criptográfica.

Exemplos:

```
# Função para gerar uma chave simétrica
def generate_symmetric_key():
    # Gera uma chave AES de 256 bits para comunicação segura
    return os.urandom(32)
```

Figura 2.15: Geração de chave simétrica utilizada para criptografia de mensagens.

```
def encrypt_data(data, symmetric_key):
    # Encripta dados usando a chave simétrica
    nonce = os.urandom(12)
    cipher = Cipher(algorithms.AES(symmetric_key), modes.GCM(nonce), backend=default_backend())
    encryptor = cipher.encryptor()
    encrypted_data = encryptor.update(data.encode()) + encryptor.finalize()
    return encrypted_data, encryptor.tag, nonce
```

Figura 2.16: Nonce do método de encriptação de dados a serem enviados.

2.4.2 Critério 6.3.2

Descrição: Garantir que GUIDs sejam gerados com um algoritmo seguro e utilizando uma fonte de entropia criptograficamente segura (CSPRNG).

Aplicação: Aplica-se ao projeto. Durante a geração de GUIDs, utilizada para criar o ID da sessão, foi usada a função `uuid.uuid4()`, que atende aos critérios necessários.

Exemplo:

```
# Gerar um ID de sessão (UUID)
session_id = str(uuid.uuid4())
```

Figura 2.17: Geração do ID da sessão com `uuid.uuid4()`.

2.4.3 Critério 6.3.3

Descrição: Garantir que os números aleatórios são criados com a entropia adequada, mesmo quando o aplicativo estiver sobrecarregado ou houver algum tipo de degradação.

Aplicação: A utilização do `os.urandom` pode ser comprometida em cargas extremas, apesar de não comprometer totalmente a segurança. O UUID v4, entretanto, é eficiente mesmo nestes casos.

Sugestão de melhorias: Implementação de um fallback seguro no caso de problemas inesperados e utilização de outra fonte geradora, como o módulo de Python `secrets`.

2.5 (6.4) Secret Management

2.5.1 Critério 6.4.1

Descrição: Verificar se uma solução de gestão de segredos, como um key vault, é usada para criar, armazenar, controlar o acesso e destruir segredos.

Aplicação:

- **Criação dos segredos:** Os segredos são criados de forma segura. Além do que foi visto na seção 6.3, o par de chaves do cliente foi criado utilizando criptografia ECC, o ID da sessão gerado de forma aleatória com UUID v4, e o par de chaves do servidor com RSA.
- **Armazenamento:** Os dados armazenados de forma criptografada incluem credenciais e pares de chaves. Houve uma falha em não criptografar outros dados, como o ficheiro de sessão.
- **Controle de acesso:** Foram implementadas verificações nas funções da API, exigindo que o utilizador tenha uma role ativa na sessão com a permissão adequada. Exemplo:

```
def add_subject(data, permission= "SUBJECT_NEW"):
    session_id = data["session_id"]

    if verify_session_file(session_id):
        session_data = load_session_from_id(session_id)
```

Figura 2.18: Primeiro é verificado se a sessão está ativa, com a (`verify_session_file`).

```

if check_permission_user(session_roles, session_org_id, session_username, permission):

    conn = sqlite3.connect('repository.db')
    create_table_if_not_exists(conn)
    cursor = conn.cursor()

    try:
        cursor.execute("""
            INSERT INTO subjects (organization_id, username, full_name, email, public_key, status)
            VALUES (?, ?, ?, ?, ?, ?)
            """, (session_org_id[1], new_username, new_name, new_email, public_key_new_user, "up"))

        #confirm and close the connection
        conn.commit()

        server_private_key = load_server_private_key()

```

Figura 2.19: Depois é verificado se o usuário tem a devida permissão com (`check_permission_user`).

- **Destruição segura:** Os ficheiros de sessão são verificados para timeout antes de serem destruídos. Uma melhoria seria realizar verificações periódicas, em vez de apenas quando um acesso é tentado.